Building a custom JS runtime in 30 minutes

Luca Casonato - Armada JS 2022

Illustrated by @hashedrock

Who am I?

Luca Casonato

Software Engineer at Deno Land Inc

Working on Deno Deploy, Fresh, and the Deno CLI

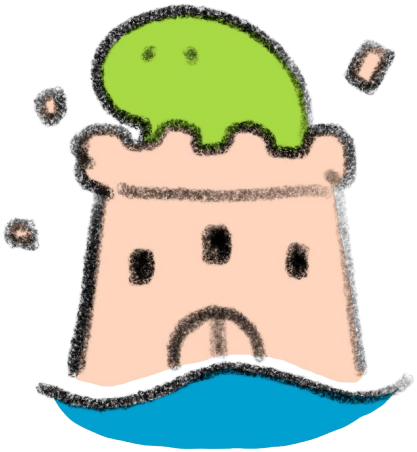
Delegate at TC39

Contributor to WHATWG and W3C specifications

Co-chair of WinterCG

What are we building?

`armadajs` - *the last JS runtime you'll need!**



- Get the version of the binary!
- Read file contents from disk!
- `fetch` data from the web!
- Runs on Mac, Linux, and Windows!

* if you do not need to actually ever use JS runtimes.

Demo

What is a JS runtime?

A system that runs JS (as specified by ECMA262).

Most runtimes provide additional APIs for IO.

Often backed by async I/O and an event loop.

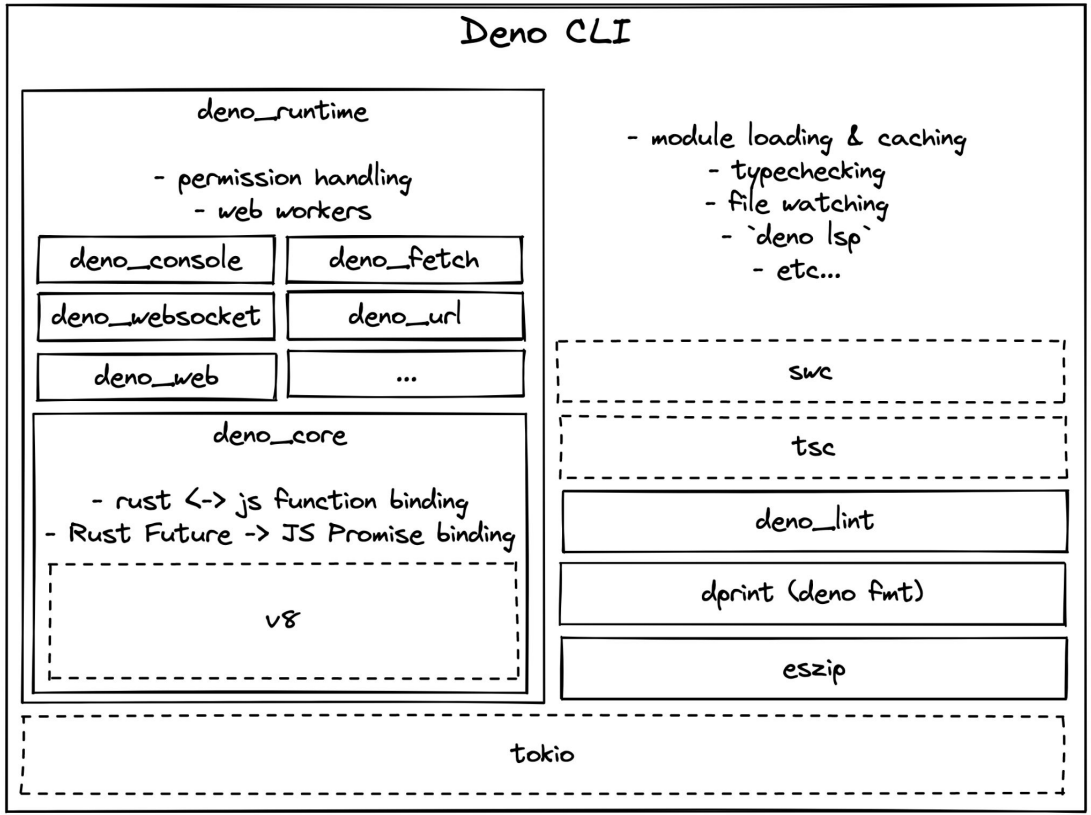
Built in a systems level language like C++ or Rust.

How Deno is built

Not a monolithic runtime.

Many layers with different responsibilities.

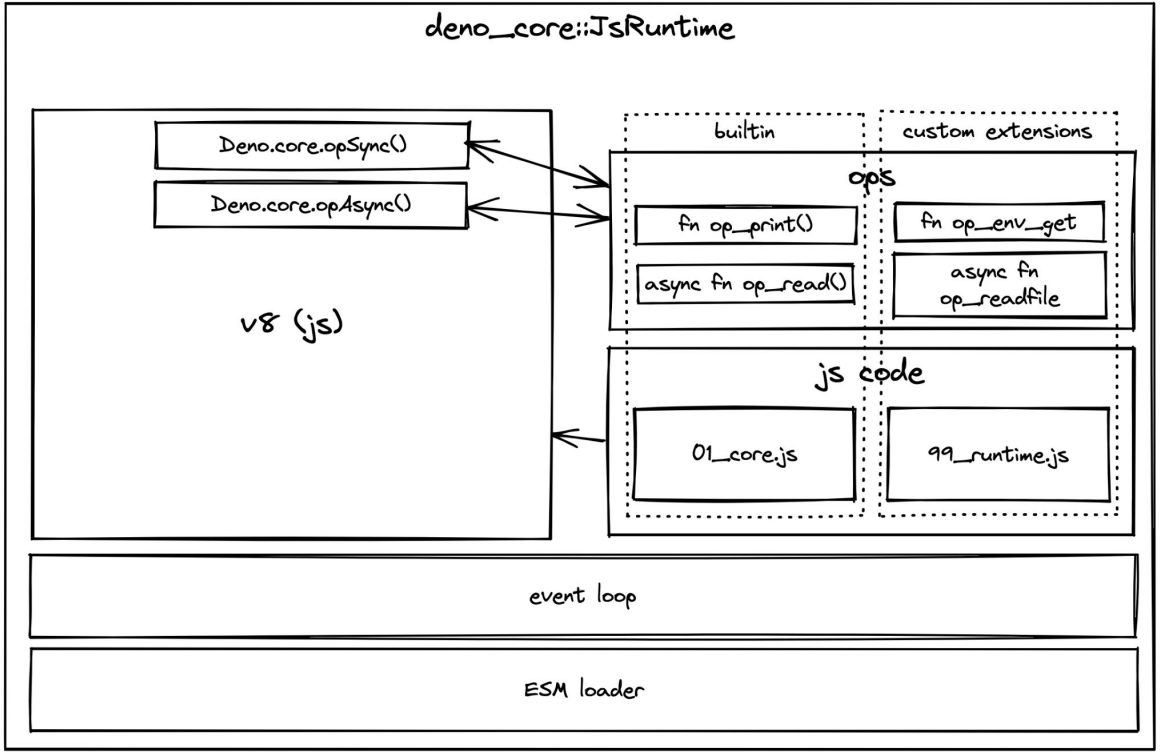
Can re-compose these layers into new runtimes.



Building `armadajs` on top of deno_core

deno_core provides essential primitives:

- Calling Rust from JS
- Binding Rust Futures to JS Promise (event loop)
- Loading JS code into V8



Code

- Read a JS file
- Create a deno_core runtime
- Execute the JS file in the runtime

Extending deno_core

Useful runtimes require additional JS APIs.

Useful additional APIs often require system calls.

System calls can only happen in Rust.

We need to be able to call Rust functions from JS.

Extending deno_core

Wrapper code surrounding raw Rust fn calls is easier to write in JS.

deno_core provides an extension mechanism: the `deno_core::Extension`

Extensions are a collection of ops (JS bound Rust functions), and JavaScript wrapper code. They are fully self encapsulated.

Code

- Create a new `deno_core::Extension`
- Add an op to get the binary version
- Define an API for it in JS

Async ops

Synchronous ops complete synchronously. No async I/O.

Asynchronous ops return a `Future` in Rust and a `Promise` in JS.

The promise resolves when the future completes.

Allows for idiomatic async operations (using `async/await`) in both JS and Rust.

Async I/O

Async I/O requires an event loop.

Unlike Rust, JavaScript has an implicit global event loop. Rust “async runtimes” are pluggable.

Tokio is a popular, battle-tested, cross-platform async I/O library and async runtime for Rust.

Code

- Create a new async op to read a file from disk
- Create a JS API to call this op
- Install and configure Tokio
- Actually drive the JS event loop from Rust

Re-using existing extensions

Deno already implements many Web APIs as `deno_core::Extension`.

These can be re-used in our runtime.

Examples:

- `console` in `deno_console`
- `fetch` in `deno_fetch`
- Streams, TextEncoder, and more in `deno_web`

Code

- Enable `deno_console`, `deno_webidl`, `deno_web`, `deno_url`, and `deno_fetch` extensions

Conclusion

Building a JS runtime from scratch is really hard work. Things that seem trivial, can be very non trivial.

With the right tools and abstractions, it can be much easier.

My prediction for the future:

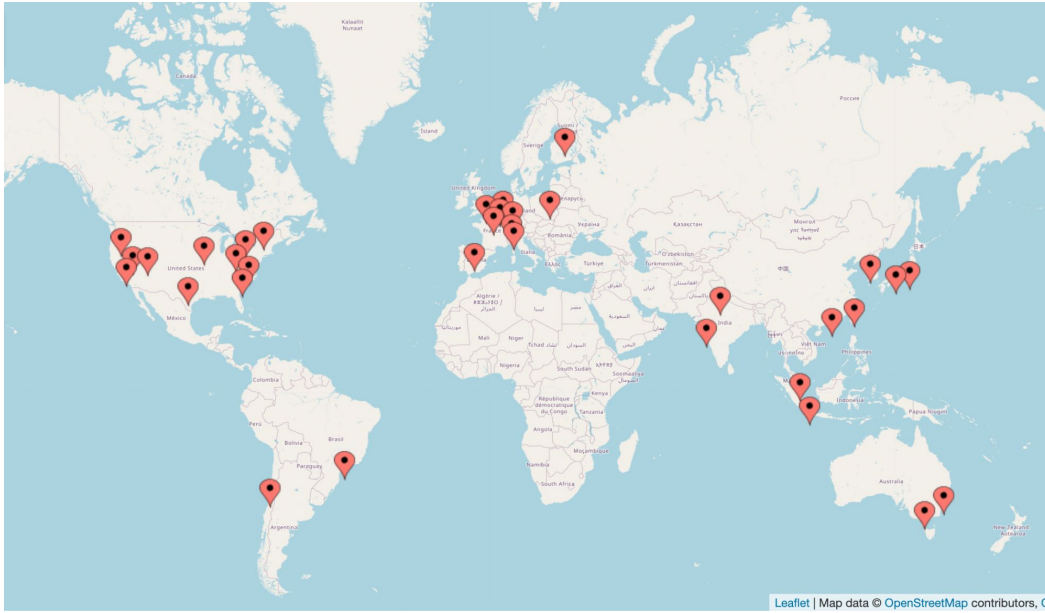
We'll see more application specific JS runtimes appear over the next few years. These runtimes will use web standards for interoperability.

A runtime built for the edge

- Deno Deploy, our hosted edge platform
  - 34 data centers
  - GitHub integration
  - Instant deploys
- Within 100ms of majority of internet users

https://deno.com/deploy

Deno Deploy powers Netlify Edge Functions



Resources

https://github.com/denoland/deno

https://deno.land

https://discord.gg/deno

https://lcas.dev/

https://twitter.com/lcasdev

We're hiring!

https://deno.com/jobs

Questions?

DM me @lcasdev or talk to me in person